

**Váci SZC Petőfi Sándor Műszaki Technikum,
Gimnázium és Kollégium**



SZOFTVERFEJLESZTŐ ÉS TESZTELŐ

Záródolgozat

Race-001 Autókölcsönző

Denkinger Ákos
Konzulens

Kucsik Gábor
13.D

2025

KONZULTÁCIÓS LAP

Szoftverfejlesztő
SZJ 50 613 1203

A hallgató neve:Osztálya:

A záródolgozat témája:

.....

.....

.....

Sorszám:	A konzultáció időpontja:	A konzulens aláírása:
1.	2025. 02. 11.	Deuliger Ákos
2.	2025. 02. 25.	Deuliger Ákos
3.	2025. 03. 19.	Deuliger Ákos

A záródolgozat beadható:
Aszód, 2025.

.....
/ konzulens/

Nyilatkozat

Alulírott tanuló kijelentem, hogy a szakdolgozat teljes mértékben a saját munkám eredménye. A felhasznált eszközöket és szakirodalmakat (tanulmány, internetes forrás, személyes közlés stb.) idézőjel és megjelölt források nélkül nem építettem be, egyértelműen megjelöltem. Az elkészült záródolgozatban található eredményeket a képzőintézmény és a feladatot kiíró intézmény saját céljára térítés nélkül felhasználhatja, a titkosításra vonatkozó esetleges megkötések mellett.

Aszód, 2024.

Tartalomjegyzék

Nyilatkozat.....	3
Tartalomjegyzék	4
Bevezetés.....	6
Felhasználói útmutató.....	7
A program általános specifikációja:.....	7
Rendszerkövetelmények:	8
A minimális hardver konfiguráció a következő:	8
Az ajánlott hardver konfiguráció:	8
Szoftverkövetelmények:	8
Egyéb szoftverek:	8
Program telepítése	9
A program használatának részletes leírása:	17
Regisztráció és bejelentkezés	19
Autóink	21
Kapcsolat	22
Rólunk.....	23
Kedvenc funkció és Saját Rendelések.....	24
Kölcsönzés	25
Fejlesztői dokumentáció.....	27
Témaválasztás indoklása:	27
Alkalmazott fejlesztői eszközök.....	28
Adatmodell leírása.....	30
Felhasználók tábla	30
Autók tábla	31
Rendelések tábla	31
Részletes feladat-specifikációk, algoritmusok.....	32
index.jsx.....	32
App.jsx	32
Backend	34
/register	35
/login	37
Vélemény API	38
Forráskód.....	39
Home.jsx.....	39

Navbar.jsx és Footer.jsx komponensek	40
Tesztelési Dokumentáció.....	41
Továbbfejlesztési lehetőségek	42
Design	42
Kedvenc funkció	42
Autók elkülönítése.....	42
Fizetés.....	42
Kölcsönzött autók.....	42
Összegzés.....	43
Források.....	44
Dokumentációk, források	44
Képek.....	44

Bevezetés

Szakedolgozatunkat Morvai Mátéval készítettük ketten. Témánk egy Luxus Autókölcsönző online weboldala. Közös érdekeltségünk az autók világa, szeretjük a letisztultságot, az egyszerűséget és a kényelmet, úgy gondoltuk, hogy ez egy tökéletes téma lehet a weboldalnak. Az oldal koncepciója a bérleten alapszik, a felhasználó széles választékból választhat kedvére autót és bérelheti azokat. Szakedolgozatunk célja a gördülékeny felhasználói élmény, valamint az egyszerű kölcsönzés. A luxusautó bérlet nálunk Magyarországon még nem terjedt el online, pedig rengeteg előnye van.

Gondolatunk az volt, hogy az általunk választott szolgáltatás nagyon kevés figyelmet kap a magyar piacon, főleg az online térben. Sokkal egyszerűbb és gyorsabb a bérlet, mivel pár kattintással le lehet foglalni az által így az érdeklődők sok időt megspórolhatnak. Ezzel a projekttel leginkább az üzletembereket, hivatalos személyeket szeretnénk volna megcélolni, és természetesen mindenki mást, akinek szüksége van egy kis tekintélyre. Szerettünk volna, hogy a mai világban, ahol az emberek szeretik a csillogást, fényűzést, egy egyszerű módot teremtsünk nekik autózás terén. Törekedtünk arra, hogy széles választék legyen a weboldalunkon, hisz nem lehet mindenkinek ugyan olyan az ízlése. Tudnak nálunk bérelni városi terepjárókat, sportautókat, kisebb-nagyobb, benzines, elektromos akár hibrid motorokkal felszerelve.

A projektünk frontend-jét HTML5, CSS, JavaScript és React segítségével építettük fel, míg a backend-et NodeJs és MySQL technológiákkal fejlesztettük. A felhasználó tud regisztrálni, bejelentkezni a saját fiókjába, és azután már bérelheti is a tetszése szerinti luxusautót. Szakedolgozatunk szerveréhez jó pár modult használtunk, ami segítségünkre volt például abban is, hogy a felhasználó adatai, jelszava teljesen biztonságban legyen adatbázisunkban. Szerettünk volna egy olyan weboldallal elő állni, ami kellemes a szemnek és örömmel használják az emberek, ez alatt a reszponzivitásra gondolok.

Felhasználói útmutató

A program általános specifikációja:

Szakdolgozatunk célja a luxusautók ismertetése és reklámozása a felhasználó számára. A weboldal ismerteti részletesebben az autók tulajdonságát, hogy a felhasználó igazán magának illő autót tudjon bérelni.

Az oldalon navigációs sávon keresztül fűleken tudunk lépkedni. A felhasználónak szüksége lesz egy fiókra, amit a jobb felső sarokban lévő hivatkozások biztosítanak. Regisztráció után feltöltődnek az adatok az adatbázisunkba, természetesen a jelszó le van védve, így nem fér hozzá senki. Regisztráció után be kell jelentkeznie, ezután teljeskörű hozzáférést kap az oldal használatához, tehát eléri a weboldal összes tulajdonságát. A navigációs sávunk 4 hivatkozásból áll, plusz a regisztráció és bejelentkezés. A **főoldal** fogad minket, ahogy elindítjuk az applikációt, ezen az oldalon vannak kártyák, autó emblémákkal ellátva, ezek a hivatkozások elvisznek minket egy olyan oldalra, ahol csak azokat az autókat listázza ki, melynek az emblémáját kiválasztottuk. A második link az **Autóink** fül, ahol az összes autót meg tudjuk tekinteni, és a felhasználó a magának megfelelőt ki tudja választani, valamint kölcsönözni tudja. A harmadik opció a **Kapcsolat**, ahol meg tudják tekinteni az elérhetőségeinket, valamint van egy e-mail-es űrlap, hogy gyorsan és effektíven tudjanak nekünk üzenetet küldeni információkkal kapcsolatban. Az utolsó link pedig a **Rólunk** oldal, ahol az oldal üzemeltetőiről és a tulajdonosokról tudnak információkat szerezni, ez alapján is eldönthetik, hogy ez a megfelelő weboldal a számukra.

Ezekon kívül a bal oldalon amint említettem van **Bejelentkezés** és **Regisztráció**, a felhasználó ezeken a fűleken tudja regisztrálni az adatait, valamint hitelesíteni a személyazonosságát. Ha mindezzel készen van, akkor lehetősége van a profilja módosítására, megtekinteni a rendeléseit és a kedvenceit. A következő képen megtalálhatóak az imént felsorolt hivatkozások. Ezek alkotják a **navigációs sávunkat**.

Race-001 Kölcsönző

Főoldal Autóink Kapcsolat Rólunk



Bejelentkezés Regisztráció

1. ábra Navigációs sáv

Ha a felhasználó bejelentkezik, akkor megjelenik még egy **Profil** és **Kosár** ikon is a **Kedvencei** mellé.

Rendszerkövetelmények:

A minimális hardver konfiguráció a következő:

- Processzor: 1 gigahertzes vagy gyorsabb processzor
- RAM: 1 GB 32 bites operációs rendszerhez, vagy 2 GB 64 bites operációs rendszerhez
- Merevlemez terület: 3 GB
- Videókártya: DirectX 10-es vagy újabb verziójával kompatibilis
- Kijelző: 1920x1080

Az ajánlott hardver konfiguráció:

- Processzor: Intel Core i5-11600 vagy Ryzen 5 5600X
- RAM: 8 GB
- Merevlemez terület: 5 GB
- Videókártya: DirectX 10-es vagy újabb verziójával kompatibilis
- Kijelző: 1920x1080

Szoftverkövetelmények:

Operációs rendszer:

- Windows 10
- Windows 11
- Linux (Modern disztribúciók pl.: Ubuntu, Fedora, Debian, Mint, Arch)

Böngésző:

- Google Chrome 109+
- Brave 1.52.x+
- Mozilla Firefox 116.0+

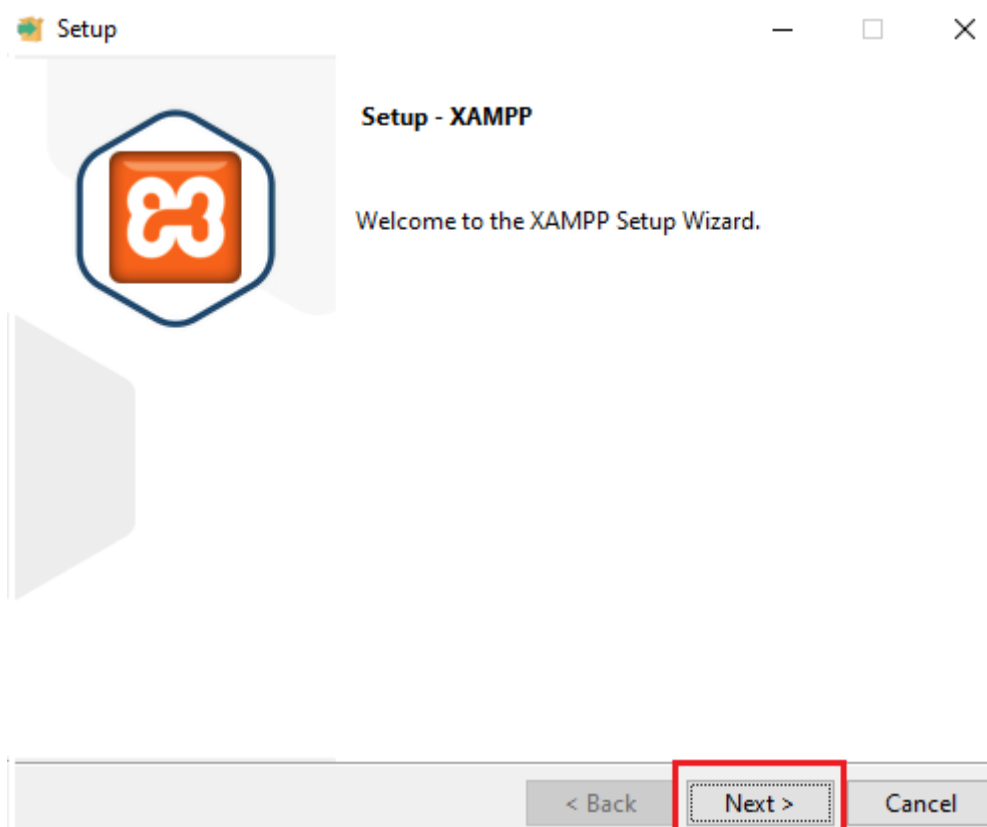
Egyéb szoftverek:

- XAMPP 8.1.10+
- NodeJs v20.3.0+

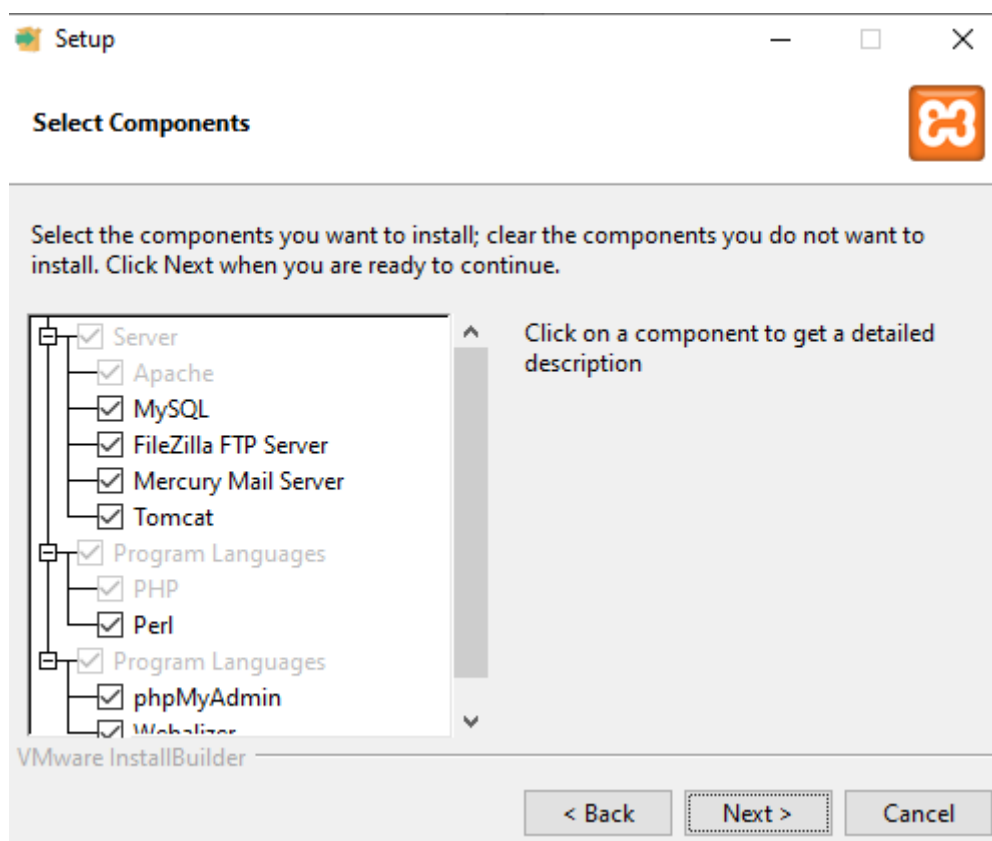
Program telepítése

Minden esetben szükségünk lesz egy böngészőre, én a szakdolgozatom elkészítése alatt **Brave**-et használtam, beépített reklámblokkolója segít mellőzni a követéseket és a nem kívánt reklámokat, valamint szintén beépített VPN (Virtual Private Network) kiegészítővel is rendelkezik. Nekem a 1.74.48-as 64 bites verzió van feltelepítve és teljesen meg voltam elégedve vele a projekt készítése alatt. A böngésző telepítőjét elérhetjük a <https://brave.com/> oldalán.

Hogy elérjük az adatbázist szükségünk lesz a **XAMPP**-ra, amit az apachefriends.org oldalán elérhetünk és le is tudjuk tölteni. Windows alatt csak elindítjuk a telepítőt és elfogadjuk a telepítési feltételeket.

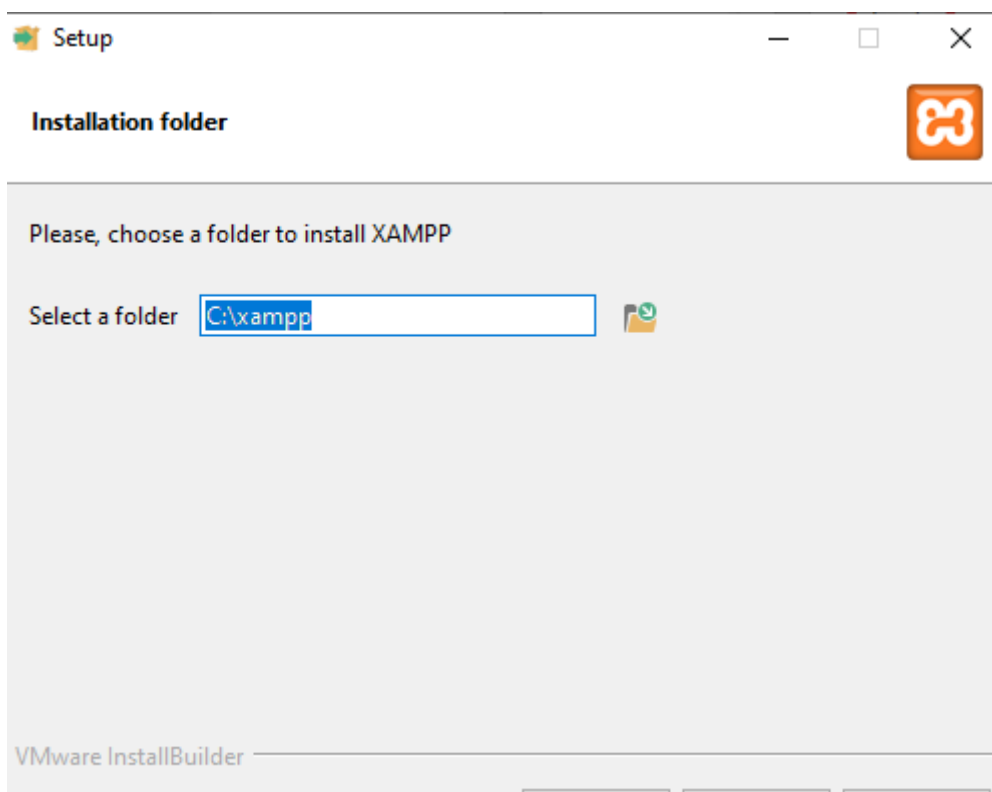


2. ábra Xampp telepítése



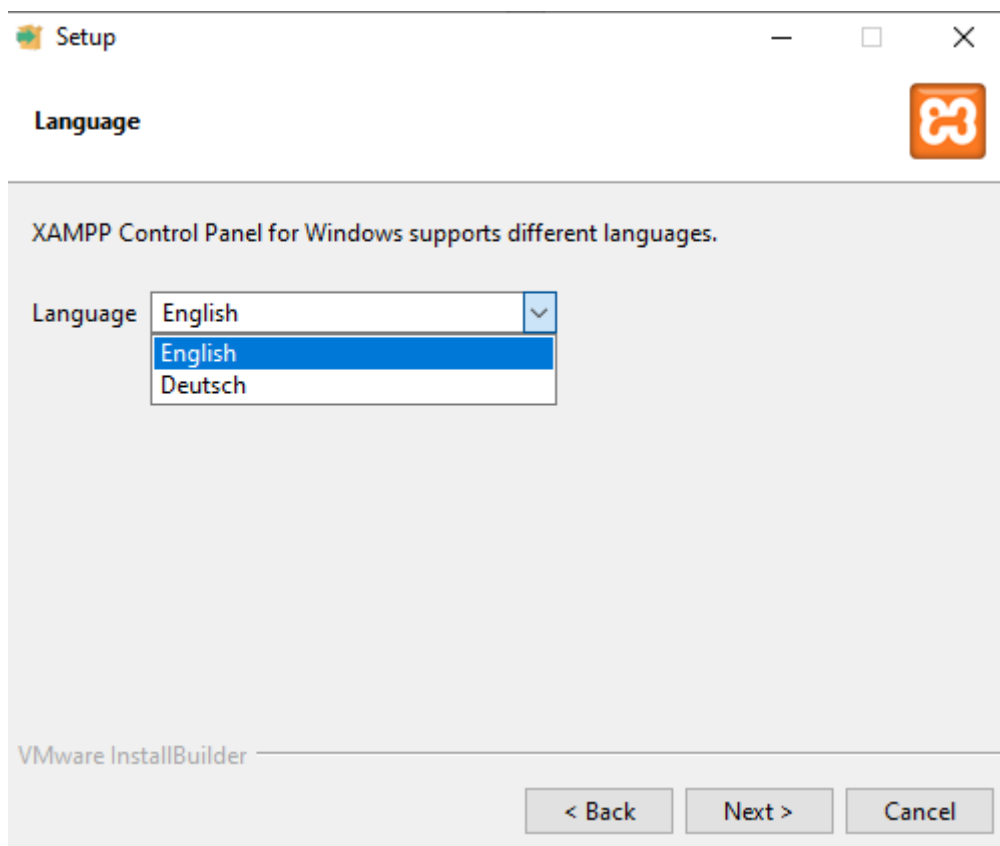
3. ábra Xampp komponensek

Itt ki kell válasszuk a szükséges komponenseket, amit a telepítő felajánl az nekünk tökéletes, így tovább mehetünk.



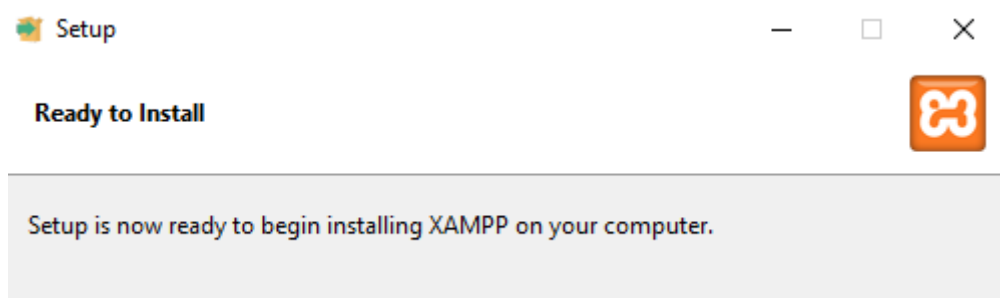
4. ábra A Xampp telepítési mappája

Saját ízlésünk szerint kiválaszthatjuk a Xampp telepítési célját, ez nem igen befolyásol semmit. A következő lépésnél kiválaszthatjuk a szoftver nyelvét, ami lehet **angol** vagy **német**. Alapértelmezett beállításként a varázsló az **angolt** ajánlja nekünk.

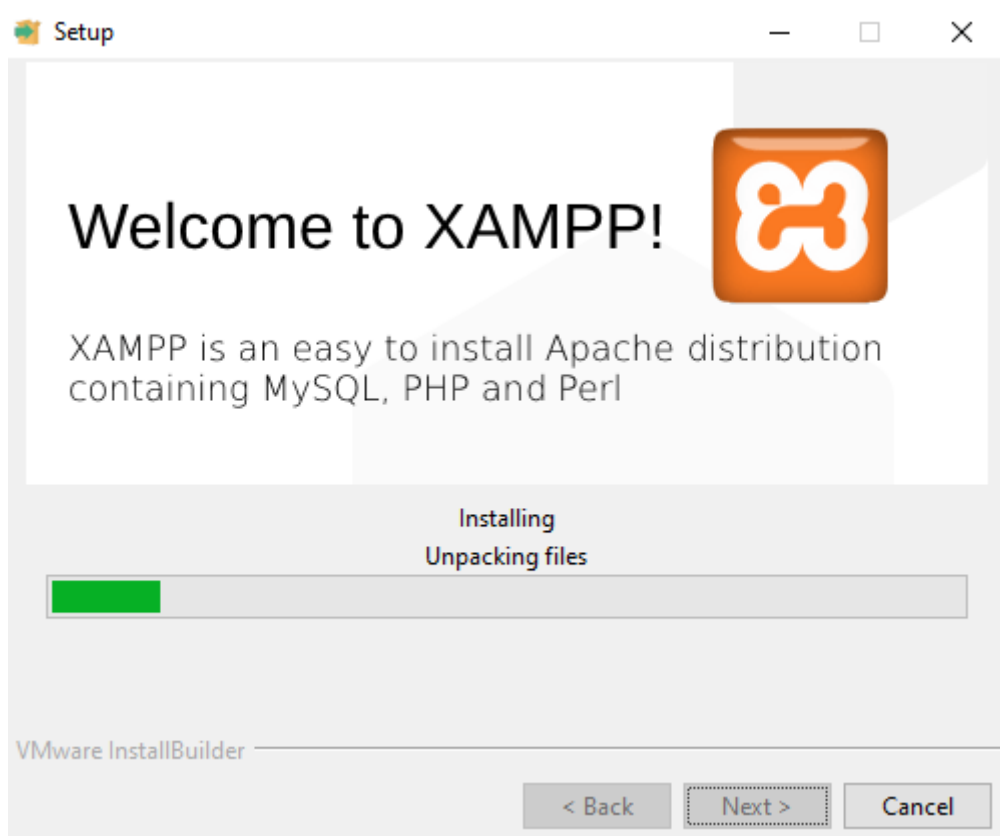


5. ábra A Xampp nyelve

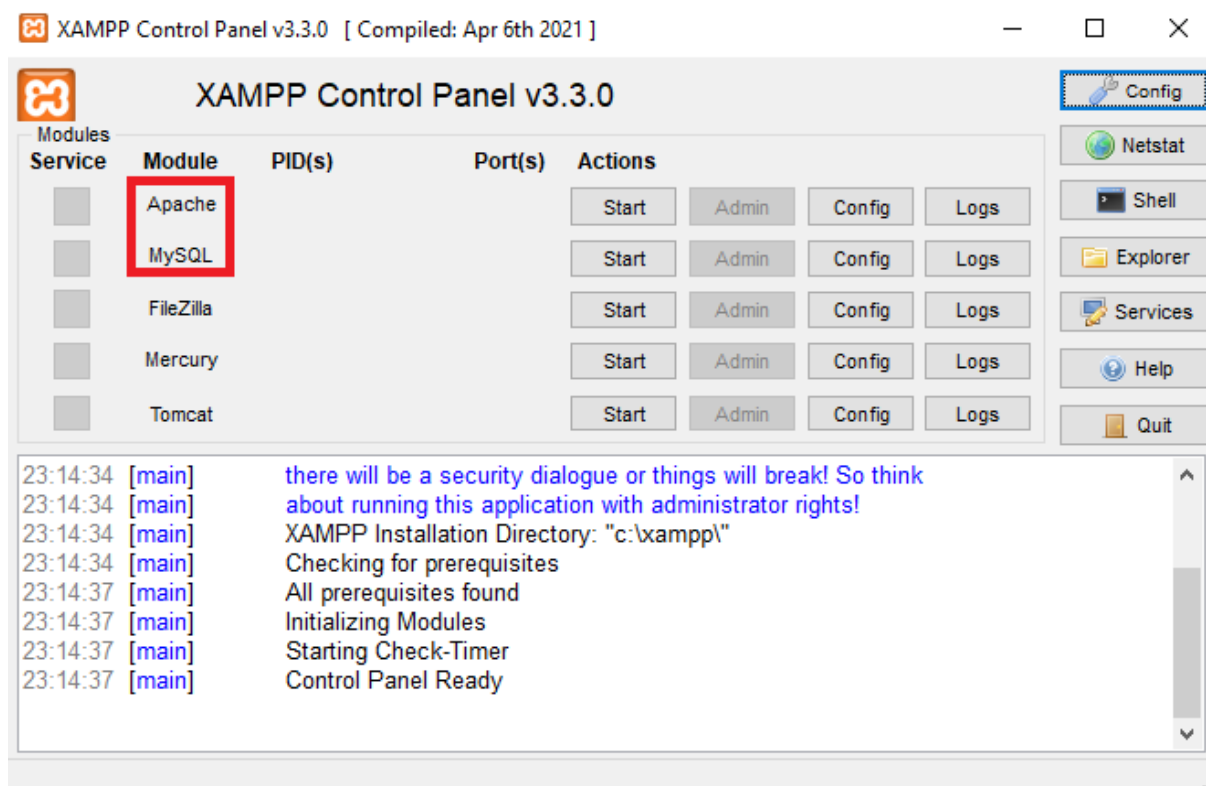
Ha kiválasztottuk a program nyelvét és rányomtunk a „Next-re” akkor a telepítő készen áll arra, hogy a szoftvert feltelepítse a számítógépünkre.



6. ábra A telepítő készen áll

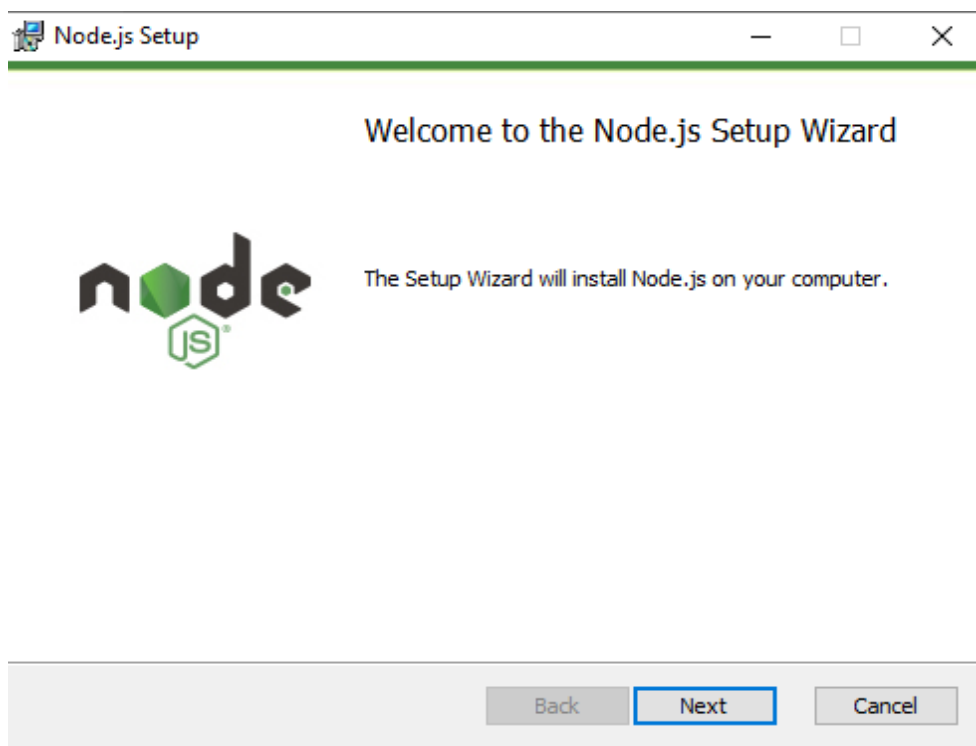


7. ábra A Xampp telepítése



8. ábra Xampp Control Panel

Következő szoftver a **NodeJs** lesz, ami kapcsolatot létesít a weboldal és az adatbázis között, és egyéb modulokat kezel, de erről később beszélek. Ezt a lépést sem ajánlott kihagyni, különben egyáltalán nem tudjuk lefuttatni magát a projektet.



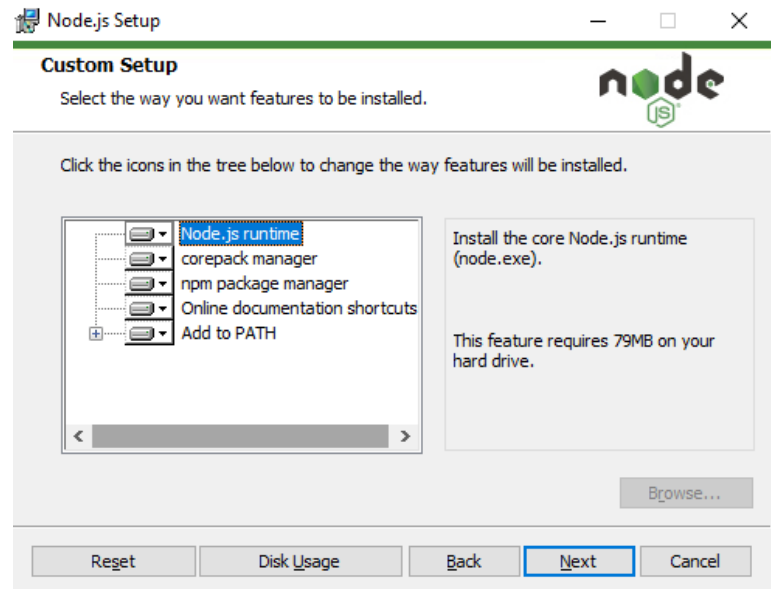
9. ábra A NodeJs telepítője

Ha tudomásul vettük, hogy a NodeJs települ a számítógépünkre akkor tovább lépünk, ezután el kell olvasni és el kell fogadni a felhasználó oldali licenz megegyezéseket, pipáljuk be azt, hogy „I accept the terms in the License agreement”.

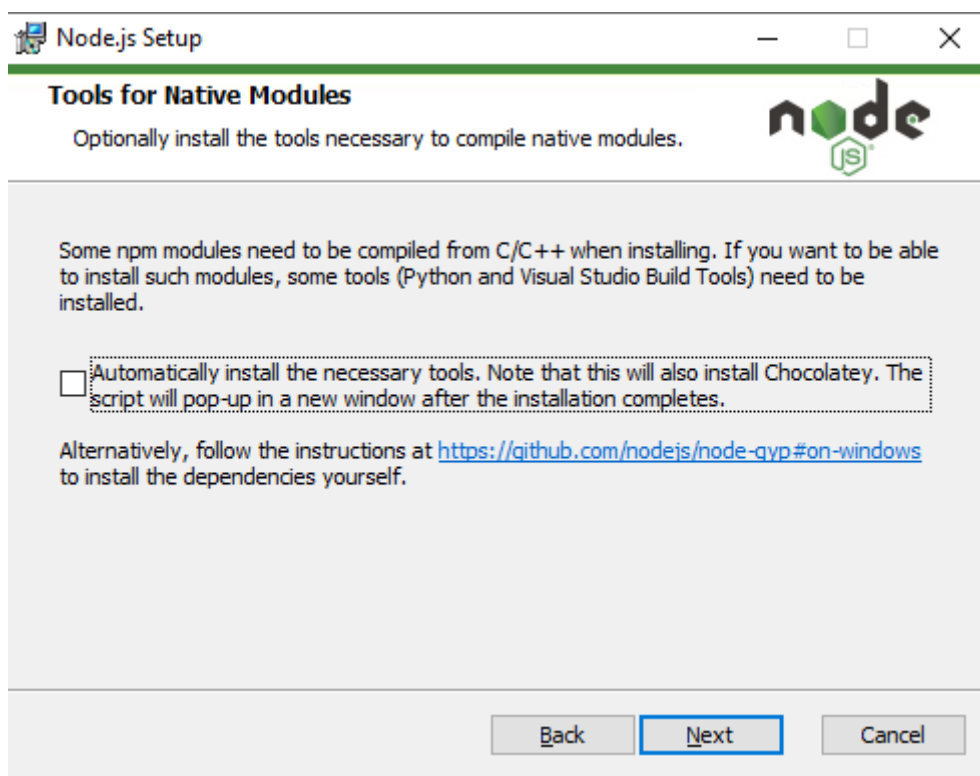
Ha elolvastuk és tovább léptünk a **Next** gombbal, akkor a telepítő megkérdezi, hogy hova szeretnénk telepíteni a NodeJs-t. Alapértelmezett beállítás itt a C:\Program Files\nodejs\. Viszont, ha más mappába szeretnénk telepíteni akkor arra is van lehetőség a **Change** gombbal.

A következő megálló a „Custom Setup”, tehát az egyéni telepítés. Itt a felhasználó ki tudja választani azokat a komponenseket, amelyeket telepíteni szeretné. Ahogy a telepítő felajánlotta az nekünk teljes mértékben megfelel, így továbbmehetünk a **Next**-el.

Ezután a varázsló megkérdezi, hogy szeretnénk-e telepíteni eszközöket natív modulokhoz. Itt csak be kell pipáljuk a rubrikát, ezáltal feltelepíti a szükséges csomagokat.

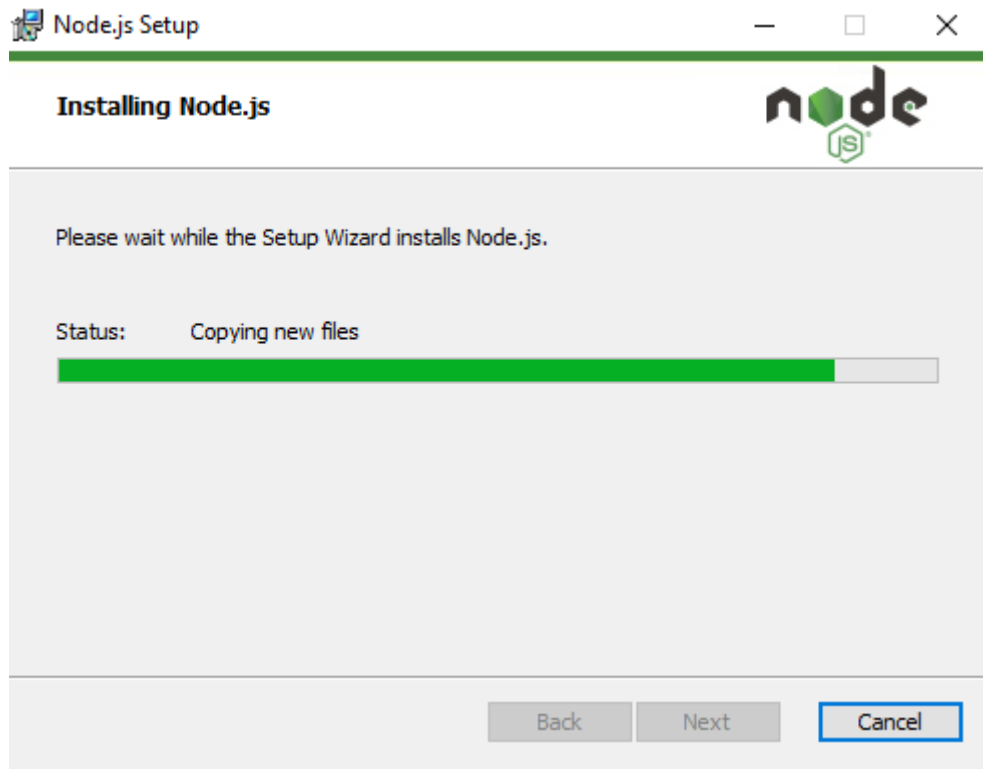


10. ábra Egyéni telepítés



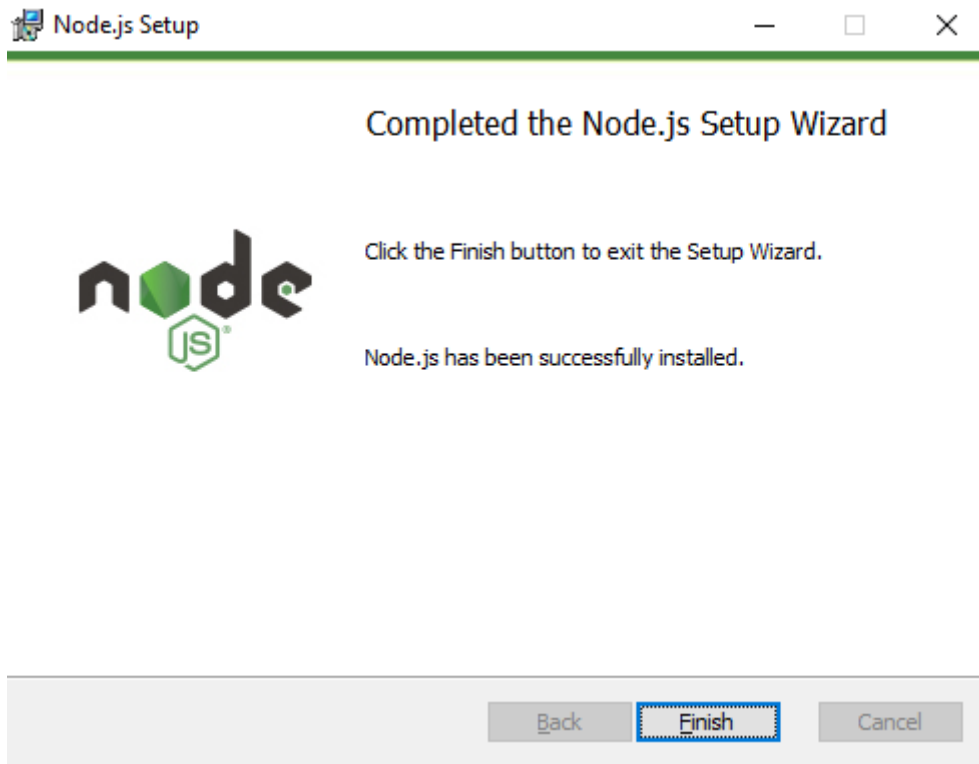
11. ábra Szükséges modulok telepítése

Ha megnyomtuk a Next-et akkor a telepítő kiírja, hogy készen állunk telepíteni a NodeJs-t, ha szeretnénk még módosítani valamit a telepítéssel kapcsolatban akkor még van rá módunk. Azonban, ha minden beállítás megfelelő számunkra akkor rá kell nyomjunk még egyszer a next-re, és el is kezdődik a telepítés. Ha végig megy a telepítés státuszát jelző csík, akkor készen vagyunk. és rányomhatunk a **Finish** gombra.



12. ábra Státusz jelző csík

Ezután feldob nekünk néhány Parancssort és Windows PowerShell ablakot, elég, ha megnyomjuk az entert minden cmd ablaknál, majd a PowerShell megkér arra sikeres telepítés esetén, hogy nyomjuk meg az Entert megint. Ezek azért futnak le, mert a szükséges eszközök, amit korábban engedélyeztünk, most települnek. A NodeJs telepítésének a végét egy Powershell ablak fogja jelezni, szimplán itt is nyomjunk egy entert.



13. ábra Sikeres telepítés

Ezennel a NodeJs telepítésével sikeresen végeztünk. Ha szükségesnek érezzük a **Visual Studio Code** telepítését, ami egy nyílt forráskódú kódszerkesztő a Microsoft által, akkor azt is feltelepíthetjük, hisz innen is el tudjuk indítani a projektet, de Windows alatt a Command Prompt, valamint Linux alatt bármelyik terminál emulátor tudja helyettesíteni a Visual Studio Code ezen funkcióját.

```
Compiled successfully!

You can now view frontend in the browser.

Local: http://localhost:3000
On Your Network: http://192. . :3000

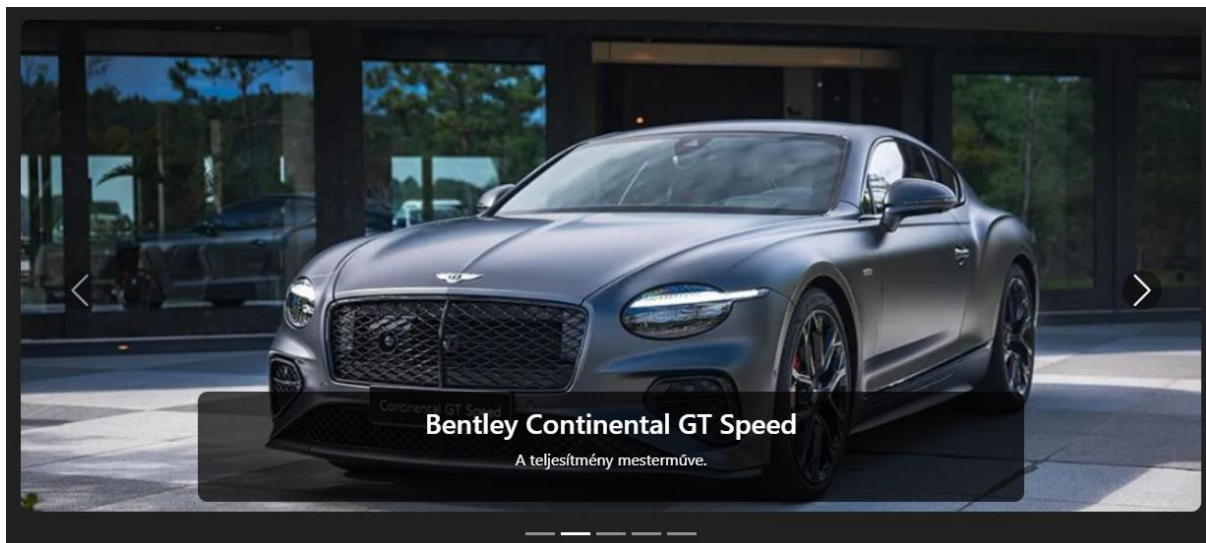
Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
█
```

14. ábra A projekt indítása Alacrittyből

A program használatának részletes leírása:

Ha elindítják a programunkat, akkor a **Főoldallal** találkoznak szembe, ami egy szlogent tartalmaz és kártyákat. Ezek a kártyák különböző márkákkal vannak ellátva, amik azt a célt szolgálják, hogy ha rájuk kattintunk akkor, csak azokat a márkájú autókat listázza ki, amelyiket kiválasztottuk. Ha lejjebb görgetünk, láthatunk egy Carousel (diavetítés szerű megjelenítés) elemet, ami képeket tartalmaz az autókínálatról. Ezzel azt akartuk elérni, hogy ha a felhasználó ránéz akkor tükrözzük az a luxust, és eleganciát egy leírással, amit kínálunk.



15. ábra Carousel elem

A navigációs sáv 3 részre van osztva, bal, közép és jobb oldal. A bal oldalon szerepel a projekt neve, ami egyben egy-egy hivatkozása főoldalra, tehát ugyanaz a szerepe, mint a „**Főoldal**” címnek. Középen az átlagos hivatkozások, amik általában mindenki számára elérhetőek, illetve jobb oldalon a bejelentkezés és regisztráció, ez megváltozik akkor, ha a felhasználó sikeresen bejelentkezett. A második hivatkozás az „**Autóink**”, ami az összes autót kilistázza a kínálatunkból, márkától eltekintve. Itt az autók kártyákban jelennek meg aminek a háttérszíne kicsit sötétebb, mint maga a háttér, ezáltal kényelmesebb a szemnek. Mindegyik autó kártyájában található egy szív emotikon, ami a kedvencé választást jelenti, így ezáltal a felhasználó a kedvenc autóit könnyen elmentheti későbbre. Emellett a funkció mellett található egy Kölcsönzés gomb is, ami egyértelműen átirányít a kölcsönzéshez. Ezt csak akkor tehetjük meg, ha be vagyunk jelentkezve, mivel a kölcsönzés gomb, a rendelés helyett a bejelentkezésre fog átirányítani. Ha továbbmegyünk a navigációs sávon akkor a „**Kapcsolat**” füllel találkozunk, ahol megtalálunk rólunk minden elérhetőséget ikonokkal ellátva, hogy tisztán érthető legyen, hogy mi micsoda. Az oldal másik funkciója egy űrlap, ahol e-mailt küldeni rögtön az oldalról. Az űrlapnak szüksége van a felhasználónévre, e-mail címre és magára az üzenetre, amiben

leírhatják az emberek, hogy miben van szükségük segítségre. Az utolsó hivatkozással elértünk a középső részlet végéhez, ez pedig a „**Rólunk**”, itt a felhasználó megismerheti az oldal üzemeltetőit egy rövid bemutatkozásban, és láthat tőlünk 2 saját kézzel készített képet kettő különböző autóról. Az oldal tetején még helyet kapott egy „**Kapcsolatfelvétel**” gomb is, ami visszairányít a **Kapcsolat** fülre.

A jobb oldali szekció pedig a kedvenc funkciót tartalmazza, valamint a **Regisztrációt** és **Bejelentkezést**. Hogy készítsünk magunknak egy profilt, a regisztrációra kell kattintanunk. Itt meg kell adnunk a saját adatainkat, mint például a felhasználónevet, teljes nevet, e-mail címet és egy jelszót. Ha ezt sikeresen teljesítettük, akkor kapunk egy üdvözlő E-Mail-t a megadott címre. Ha ezzel meg vagyunk akkor az oldal átirányít minket a Bejelentkezésre, ahol hitelesítenünk kell magunkat a felhasználónév és jelszó megadásával. Ha ezt is teljesítettük, akkor megnyilvánul előttünk a navigációs sáv a teljes formájában. Lehetőségünk van a profil szerkesztésére, és **megtekinteni a kölcsönzéseinket**, mindezek mellett élénk táru egy **Kijelentkezés** gomb is, ami egyértelműen kijelentkezteti a felhasználót. A **Profil** fülre nyomva megtekinthetjük adatainkat és szerkeszthetjük azokat.

Profil szerkesztése	Profil adatai:
Felhasználónév Új felhasználónév E-Mail cím Új E-Mail Jelszó Új jelszó Mentés	 ID: 1 Felhasználónév: gabor Teljes név: Kucsik Gábor E-mail cím: kucsikgabor22@gmail.com

16. ábra Profil oldal

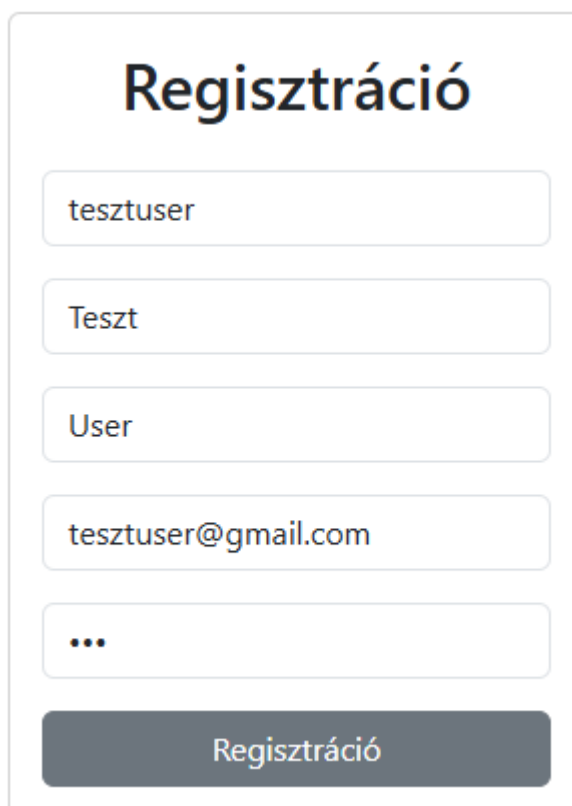
Szerettük volna, hogy egyszerű, de mégis nagyszerű maradjon a weboldal témája, így ennyiből tevődik össze a projektünk. A következő pár oldalon bemutatom és demonstrálom, hogy hogyan is működik pontosabban a weboldal, hogyan kell használni.

Regisztráció és bejelentkezés

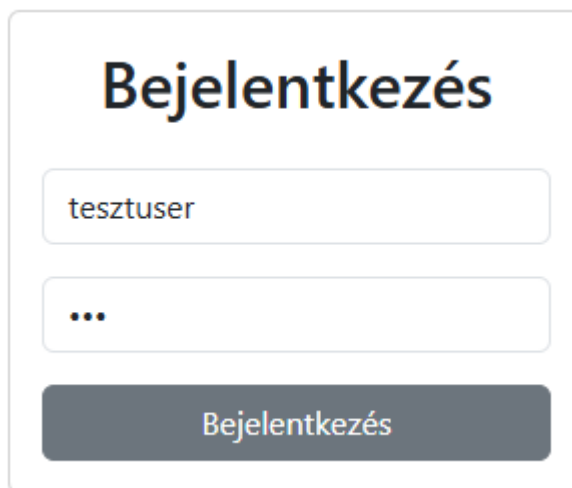
Egy új felhasználót kell regisztrálnunk a Regisztráció menüpont alatt. Szükségünk lesz egy felhasználónévre, teljes névre, e-mail címre és természetesen egy jelszóra. A megadott E-Mail címre egy üdvözlő levél fog érkezni, ami köszönti önt, mint új felhasználót a weboldalon. A regisztrációs felület így néz ki, a rubrikák üres állapotban egy helykitöltő szöveget tartalmaznak, így nem lehet elrontani a regisztrációs folyamatot azzal, hogy valamit rossz helyre írunk. Ha mindent kitöltöttünk helyesen akkor rányomhatunk a „**Regisztráció**” gombra.

Ezek után az oldal átnavigál minket a Bejelentkezésre, mivel szükséges magunkat hitelesíteni, a felhasználónév és jelszó megadásával. Egyszerűen csak megadjuk ezt a két adatot és egy gombnyomásra vagyunk attól, hogy használjuk a friss fiókunkat. A „**Bejelentkezés**” gomb lenyomásával visszairányít minket a főoldalra.

Ha ezekkel megvagyunk akkor a navigációs linkek kibővülnek három új opcióval. A **rendelésekkel**, **profil szerkesztésével** és a **kijelentkezéssel**. A rendelések menüpont nagyon hasonló a kedvencekhez, mivel mind a kettő legördülő menü. Ez azt jelenti, ha rányomunk, akkor nem visz át másik oldalra, hanem megjelenik az ikon alatt egy kis tér, ahol megtekinthetjük ezekben a kedvencek tárolt autóinkat, illetve a rendeléseket.

A regisztrációs űrlap a "Regisztráció" címmel. Tartalmaz négy szövegmezőt: az elsőben "tesztuser", a másodikban "Teszt", a harmadikban "User", a negyedikben "tesztuser@gmail.com". Alatta van egy három pontot tartalmazó mező. Az űrlap alján egy sötét szürke gomb található, amelyen a "Regisztráció" felirat olvasható.

17. ábra Regisztrációs űrlap

A bejelentkezési űrlap a "Bejelentkezés" címmel. Tartalmaz két szövegmezőt: az elsőben "tesztuser", a másodikban három pontot. Az űrlap alján egy sötét szürke gomb található, amelyen a "Bejelentkezés" felirat olvasható.

18. ábra Bejelentkező űrlap

Most, hogy regisztráltunk, szabad kezét kaptunk a kölcsönzéshez is, mivel hogyha a felhasználó nem bejelentkezett állapotban szeretne kölcsönözni, akkor ez a hibaüzenet fogadja.

A(z) localhost:3001 közlendője

Kölcsönzéshez be kell jelentkezned!

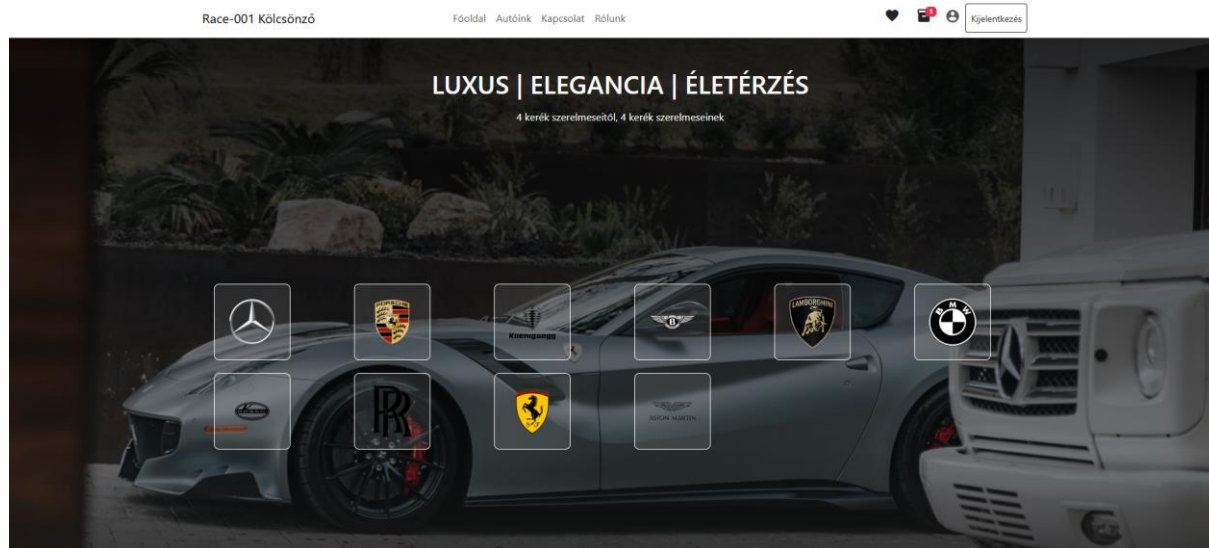


19. ábra Hibaüzenet a kölcsönzésnél

Főoldal

A főoldalon találhatunk egy szlogent, miszerint a koncepciónk a **Luxus, elegancia** és az **életérzés** körül forgott, valamint ez alatt egy kis bekezdéssel találkozhatunk, ami azt sugallja a felhasználónak, hogy közősek lehetnek érdeklődési köreink, illetve hobbijain. Ezáltal hátha növekedik benne az az érzés, hogy minket választ.

Mindezek alatt elhelyeztünk 10 gombként funkcionáló kártyát, amik autók márkájával vannak ellátva. Ezek a gombok azt tudják, hogy márka szerint szétválogatva kilistázza az autóinkat. Ezek az oldalak ugyanarra a témára épültek, mint az **Autóink** oldal.



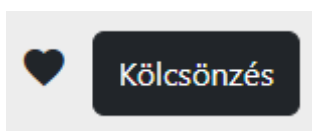
20. ábra Főoldal

Ha lejjebb görgetünk találunk egy kis ízelítőt a választékunkból, ami egy diavetítés szerű elem képekkel ellátva, amit tudunk lapozni. A képek elvannak látva az autó márkájával, modell nevével, és egy rövid leírással, ami bemutatja, hogy mire számíthatunk az adott járműtől, ezek a szövegek el vannak látva egy enyhén átlátszó fekete dobozzal a jó átláthatóság

érdekében. A két szélén találhatunk nyilakat, ami arra szolgál, hogy előre-hátra tudjuk lapozni a bemutatót.

Autóink

Az Autóink oldal kártyákban tartalmazzák az autó márkáját, modelljét, évjáratát, motor adatait és árát. A márkajelzés alá van húzva, hogy egyszerűbb legyen felismerni az adott autót. A kártyában helyet kapott még egy kedvenc és egy kölcsönzés gomb is. A **kedvenc** gombot egy szív ikon körvonala ábrázolja, amit hogyha megnyomunk kitöltődik a szív, ami azt jelenti, hogy bekerül a navigációs sávon található kedvenc menüpont alá egy legördülő menüben. A **kölcsönzés** gomb pedig egy egyszerű gomb, ami átirányít egy oldalra, ahonnan ki tudjuk bérelni az adott járművet.



22. ábra Autó kártya gombok aktivált állapotban

Az autók egy oldal úgynevezett „masonry” stílust kapott, ami azt jelenti, hogy a kártyák úgy állnak egymáshoz, mint egy építkezésen a téglák. Ezáltal kényelmesebb a szemnek és elegánsabban néz ki az oldal.



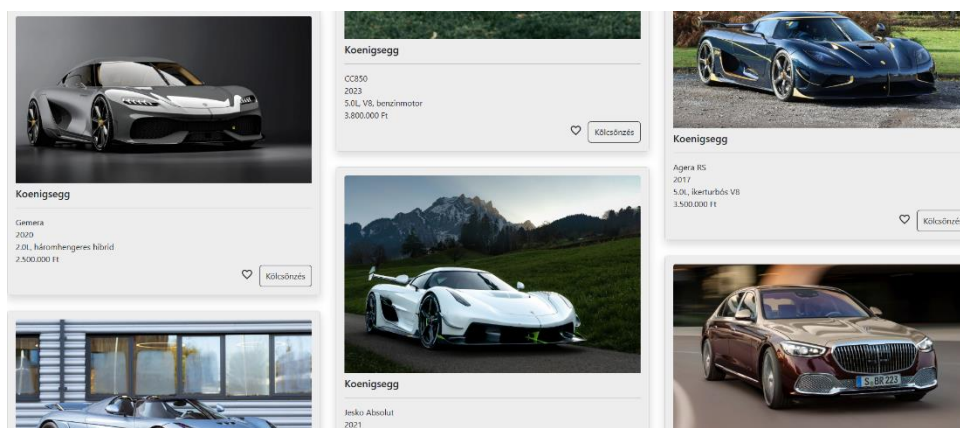
Mercedes-Benz

S-Class
2021
4.0L, V8, benzinmotor
140.000 Ft



Kölcsönzés

21. ábra Autó kártya példa



23. ábra Masonry stílus

Kapcsolat

A **kapcsolat** oldalon láthatjuk az oldal címét aláhúzva, ezalatt pedig két másik bekezdést, ami tudatja a felhasználóval, hogy itt tudja felvenni velünk a kapcsolatot az ügyfélszolgálat, ami 0-24 elérhető.

Az oldalon két résszel találkozhatunk, az egyik egy e-mail küldő űrlap, ahol a felhasználó üzenetet tud küldeni a fejlesztőknek, ha valami problémába ütközik, vagy kérdése van. Ez a részleg 3 mezőből áll, felhasználónév, e-mail cím (amiről szeretnénk küldeni a levelet) és természetesen az üzenet, ahova a kérést, kérdést lehet írni. Ezután a küldés gombbal elküldhetjük az oldal üzemeltetőinek.

Felhasználónév

E-Mail cím

Üzenet

Küldés

24. ábra Űrlap a levél küldéséhez

Az oldal jobb oldalán pedig azok az adatok találhatóak meg, amiken keresztül tudnak nekünk írni, illetve az üzlet fizikai címe. Ezek ikonokkal vannak ellátva, hogy könnyebben el tudjon igazodni a felhasználó az információk között.

Rólunk

Ezen az oldalon helyet kapott a kölcsönző neve, egy alcímmel, ami tudatja a felhasználóval, hogy itt tudja jobban megismerni a fejlesztőket. Ide raktunk egy Kapcsolatfelvétel gombot is, ami

Race-001 Autókölcsönző - Rólunk

Ismerd meg jobban a csapatunkat

Kapcsolatfelvétel

Írtunk egy rövid

25. ábra A Rólunk oldal címe

bemutakozó szöveget, ami két részlegre van osztva, mind a kettő mellett helyet kapott egy-egy saját kézzel készült kép. Mindkét szöveg kezdőbetűje kapott egy iniciálét, aminek arany színe van, valamint ezek a bekezdések a képekkel egy szürke árnyalatú dobozban vannak.

Üdvözlünk a Race-001 Autókölcsönző weboldalán! Mi egy két fős csapat vagyunk, akiknek közös szenvedélye az autók világa. Hiszünk abban, hogy egy igazán különleges autó nem csupán egy közlekedési eszköz, hanem egy élmény, amely minden utazást felejthetlenné tesz. A mi gondolatunk a kényelem, az elegancia és a luxus köré épül. A célunk, hogy a legjobb prémium autót kínáljuk ügyfeleinknek, akik értékelik a minőséget és a stílust. Minden autót, amit kölcsönözhetek tőlünk, úgy válogattunk, hogy a legmagasabb szintű vezetési élményt biztosítsa, miközben minden utazás kényelmes és emlékezetes marad. Csapatunk elkötelezett abban, hogy a legjobb autós élményeket nyújtsa számodra, és segítsen megtalálni az ideális járművet bármilyen alkalomra.



26. ábra Első szövegdoboz



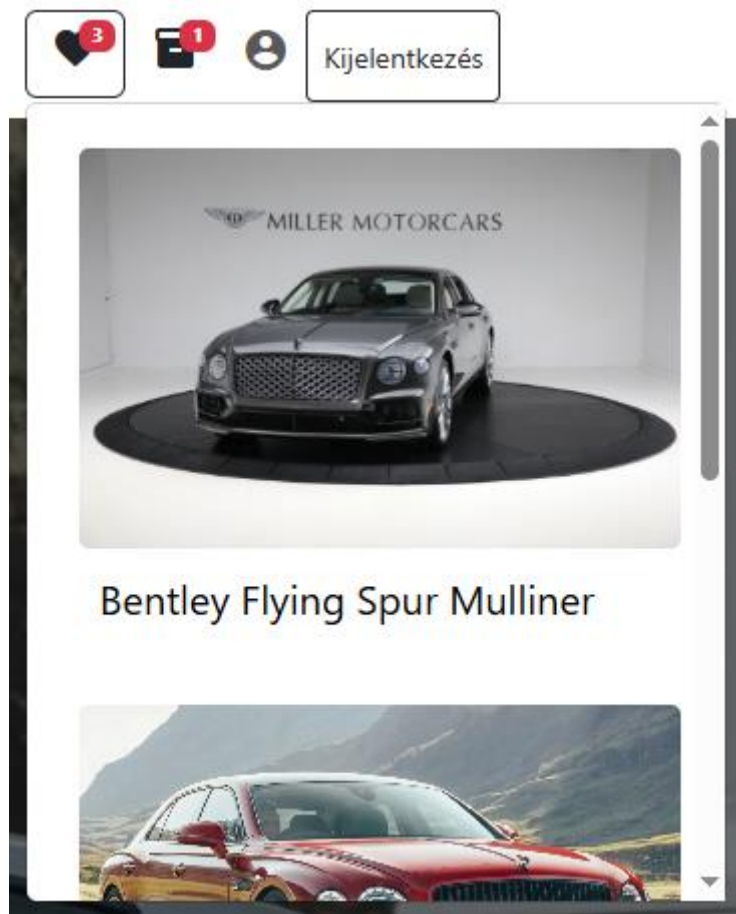
ARace-001 Autókölcsönző weboldalán rengeteg prémium autó márká közül válogathatsz, így biztosan megtalálod azt a járművet, amelyik a legjobban illik hozzád. Legyen szó egy kényelmes városi autóról, vagy egy elegáns sportkocsiról, nálunk mindenki talál számára megfelelő opciót. Mi mindig a legjobb minőségre törekszünk, hogy az autózás élménye valóban különleges legyen. Emellett az ügyfélszolgálatunk 0-24 órában elérhető, így bármikor kereshetsz minket, ha kérdésed van, vagy segítségre van szükséged. Az adatkezelés kiemelt figyelmet kap nálunk, mindent megteszünk annak érdekében, hogy a személyes adataid teljes biztonságban legyenek. Mi biztosítjuk, hogy minden információ védve legyen, és nyugodtan használhatod a szolgáltatásainkat anélkül, hogy aggódnod kellene.

27. ábra Második szövegdoboz

Kedvenc funkció és Saját Rendelések

Ezekon a részekon lesz megjelenítve az Autóink oldalról megjelölt, illetve kölcsönzött autók. Ez a két menüpont egy legördülő menüpont a navigációs sávon. Egy személy csak egy autót tud kölcsönözni egy időben, míg a kedvencek közé annyi autót tud megjelölni, amennyit csak szeretne.

Mind a két menüponon egy piros háttérű jelvény jelzi, hogy hány elem van a menüben. A kedvenceknél ez az a szám lesz amennyit megjelöltünk, viszont rendelések esetén mindig egy lesz. A **szívecske** ikont, ha lenyitjuk akkor egy görgetősávval tudunk navigálni a kedvenceink között. Az autóról megkapjuk a képet, márkáját és a nevét, hogy a későbbiekben könnyen tudjunk tájékozódni, ha kölcsönözni szeretnénk. A **rendelések** menü kibővült egy piros „törlés”, és egy zöld „meghosszabbítás” címmel ellátott gombbal. Egyértelműen a törlés gomb törli a rendelt, míg a meghosszabbítás meghosszabbítja a kölcsönzés időtartamát.



28. ábra Kedvenc menü

Kölcsönzés

Kölcsönözni az előbb említett „Kölcsönzés” gombbal tudunk, ez átirányít egy oldalra, ahol át tudjuk nézni a kiválasztott autó adatait, és mellé kapunk egy kitöltésre váró űrlapot, ahova a saját adatainkat kell feltölteni.

Az autó adatai címsor alatt megtekinthetjük még egyszer az autó márkáját, modelljét, évjáratát, motorjának adatait, valamint az árát. A márka és modell jól el lett különítve a többi adattól, hogy jól átlátható legyen, és biztosra tudjon menni a felhasználó.

Az említett űrlap öt beviteli mezőből áll, ezek a **teljes név** (vezetéknév és keresztnév), **e-mail cím**, **telefonszám**, **szállítási cím**, **megjegyzés**, a megjegyzésen kívül az összeset kötelező kitölteni. Megszerettük volna könnyebbíteni az ügyfél dolgát azzal, hogy automatikusan kitöltjük helyette a teljes név, és e-mail cím mezőket, mivel ezeket már megadta korábban mikor felhasználói fiókot regisztrált. A rendelésünket a **küldés** gombbal küldhetjük el. Fontos tudnivaló, hogy **egy névvel csak egy autót lehet kölcsönözni**.

Ha biztosra mentünk az autó választásával, és kitöltöttünk minden adatot akkor rá is nyomhatunk hogy küldés, ezután kapunk egy értesítést zöld színnel ellátva, hogy a rendelés feldolgozás alatt áll, ugyanebben az értesítésben található egy főoldalra mutató linket, ennek megnyomásával nézelődhetünk tovább az oldalon.

Autó Részletei	Szállítási adatok
 Bentley Flying Spur V8 Évjárat: 2021 Motor: 4.0L, V8, benzinmotor Ár: 210.000 Ft	Vezetéknév és keresztnév Kucsik Gábor E-mail cím kucsikgabor22@gmail.com Telefonszám 06702305803 Szállítási cím 2116, Zsámbok Széchenyi utca 16. Megjegyzés* Küldés

29. ábra Autó kölcsönzése

Rendelése feldolgozás alatt áll.
[Vissza a főoldalra.](#)

30. ábra Értesítő sikeres rendelés esetén

Fejlesztői dokumentáció

Témaválasztás indoklása:

Témaválasztásunkhoz közre játszott a mindennapi közlekedés, és azok a csillogó autók, amivel elhaladnak mellettünk ismeretlenek, hisz akarva, akaratlanul megnézzük őket, bármilyen autóról is legyen szó. Nincs olyan ember, aki ne szeretne kényelmesen utazni, itt jött össze az autóbérlés gondolata a luxussal.

Ezeknek az autóknak a bérlése egyre népszerűbb, mivel nagyon sok üzletember, sportoló, híresség és még sorolhatnám, élnek ezzel a villogó lehetőséggel. Vannak események, ahol megfordul egy-egy ilyen bérlés, hisz úgy vannak ezekkel az alkalmakkal, hogy nagyon ritkán van ilyen az életben. Ez egyfajta státuszsimbólum, amely azt sugallja a kívülálló embereknek, hogy a bérlő miket ért már el az életben, viszont a reakciók lehet megoszlóak lesznek. Manapság egyre elterjedtebb a fiatalok körében, hogy felveszik videóra és feltöltik az internetre, ez egyfajta reklám az ilyen cégek számára. Itt jött az ötlet, hogy ha már az emberek az interneten látják ezeket az úgymond „reklámokat”, akkor azt ott helyben ki is tudják kölcsönözni, ezáltal ne kelljen órák hosszát utazni és sorban állni egy ilyen szalonban. Kényelmesen el tudják intézni otthonról, böngészhetik a kínálatot anélkül, hogy el kellene hagyniuk a házat.

Így emellett a téma mellett döntöttünk, hisz mi is sokszor felkapjuk a fejünket, ha egy hatalmas presztízzsel rendelkező luxusautó elhalad mellettünk az úton. Ami a dizájnt illeti, szerettünk volna tovább haladni ezzel a fényűző stílussal, így ez alapján választottuk meg az oldal kinézetét, betűtípusát stb.

Alkalmazott fejlesztői eszközök

- Brave – Weboldal megnyitása és tesztelése
- Visual Studio Code – Kód szerkesztése
- XAMPP – Adatbáziskezelés, lokális webszerver
- Postman – API-ok tesztelése
- Photoshop – Képek szerkesztése
- GIMP – Szintén képek szerkesztése
- NodeJs – Webszerver készítése
- GitHub Desktop – Tárolás a felhőben
- Windows képmetsző – Képek kivágása
- Word 2021 - Dokumentálás

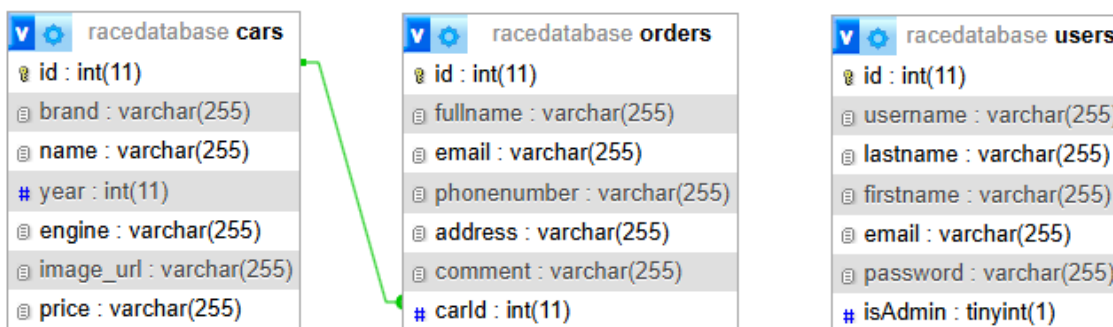
Ezek közül a programok közül néhányról már volt szó a Program telepítése cím alatt, mint például a **XAMPP**-ről, ami egy helyi webszerverként használható környezet és **NodeJs**-ről, amely lehetővé tette a szerveroldali JavaScript kódok futtatását a záródolgozatom elkészítése alatt.

A kód szerkesztéséhez **Visual Studio Code**-ot használtam, ami nagyon hasznos volt a projekt elkészítése alatt, mivel számos beépített bővítményt használhatunk, ami segítségünkre lehet a kód fejlesztése közben. Szerintem egyik legjobb tulajdonsága a hibakereső, sokszor segítségemre volt a hibák javításán, egyszerű és könnyen értelmezhető. A megírt API kéréseket **Postman**-ben teszteltem. Ez egy egyszerű vizuális eszköz, ami lehetővé teszi a kérések tesztelését. Képek szerkesztéséhez, kivágásához a következő 3 programot használtam: **Photoshop**, **GIMP** és a **Windowsos képmetsző**. A Photoshop és a GIMP egyszerű képszerkesztési lehetőségeket nyújtanak, de ha már az egyszerűségről van szó akkor inkább a GIMP, hiszen könnyű beszerezni Windowsra, Linuxra és macOS-re is, míg a Photoshop fizetős. A képmetsző pedig egy beépített eszköz a Windowsos rendszerekben, ami lehetővé teszi, hogy egy gyors billentyűkombinációval képernyőképet csináljunk.

A **GitHub**-ot pedig a projektem tárolására és megosztására használtam. Ez az eszköz lehetővé teszi a változtatások megtekintését a kód fejlesztése közben, sokan ezért is részesítik előnyben ezt a verziókezelő rendszert.

Adatmodell leírása

Ezalatt a cím alatt a weboldal adatbázis felépítését fogom bemutatni, ami fontos szerepet játszik a projekt működésében, mivel itt tárolódnak el a felhasználók és az autók adatai. Maga az adatbázis három táblából áll. ezek a felhasználók, autók és a megrendelések. Azt még hozzá szeretném tenni, hogy az `image_url` mezők tartalmazzák az autók képeit a „cars” táblában, ezek nem a saját képeim. Ezért fel szeretném hívni arra a figyelmet, ha a `database.sql` fájlt megnyitjuk, akkor megnézhetjük a képek forrásait, weboldalait, ahonnan szereztem a képeket.



31. ábra Adatbázis szerkezete

Az adatbázis összetételéhez három adattípust használtunk, ezek az `int`, `varchar` és a `tinyint`. Az `int` típusú mezőkben számokat tárolunk, a `varchar`-ban szöveget, pontosabban 255 karakter hosszút, míg a `tinyint`-et eldöntésre. Tehát ha a `tinyint`-ben 1-es szerepel akkor igaz, ha 0 akkor hamis. Az autók tábla össze van kapcsolva a rendelések táblával az `id`-n keresztül inner join-nal. Mivel szükségünk volt arra, hogy ha a felhasználó autót kölcsönöz, akkor az autó azonosítóját elmentsük a rekordban, ezáltal megkönnyebbítve a későbbi munkánkat, vagy a továbbfejlesztés lehetőségét.

Felhasználók tábla

Ez a tábla tartalmaz egy `AUTO_INCREMENT`-el ellátott **id** nevű mezővel, ami ugyanakkor a tábla elsődleges kulcsa. Az `AUTO_INCREMENT` attribútum biztosítja azt, hogy a szám (az `id`) folyamatosan növekedjen és ne legyen kettő ugyanolyan azonosító. A további mezők, azok a **username** (felhasználónév), **lastname**(vezetéknév), **firstname**(keresztnev), **email**, **password**, valamint az **isAdmin**. Mint az ábrán láthatják fentebb, ezek közül az összes `VARCHAR` típusú mező 255 karakteres hosszúsággal, kivéve az `id`-t és az `isAdmin`-t. Az `isAdmin` egy `tinyint` típusú mező, amit egy logikai eldöntésre használtunk, aminek az a lényege, hogy a felhasználónak vannak-e admin jogosultságai, vagy sem.

Autók tábla

A tábla önmagában 7 oszlopot tartalmaz. Hasonlóan a felhasználók táblához, itt is van egy **id**, ugyanazokkal az attribútumokkal, csak ez az autók azonosítóját határozza meg. A következő mezők pedig: **brand** (az autó márkája), **name** (az autó modellje), **year** (évjárat), **engine** (motor adatai), **image_url** (megjelenített képnek a címe) és **price** (az autó ára). A mezők, az id és a year kivételével szintúgy VARCHAR típusúak és 255 karakter hosszúságúak. Még azt hozzátenném, hogy a price mező magába foglalja a valutát is, nem csak az árat, ezért lett szöveg, így egyértelműbb, hogy mivel számoljon a felhasználó.

Rendelések tábla

Az utolsó táblánk szintén 7 oszlopból áll, találhatunk benne szintén egy id-t, ami ugyanúgy AUTO_INCREMENT, tehát minden rendelésnek megvan a saját azonosítója. A többi mező a felhasználó adataiból áll, ebből kettőt az oldal automatikusan kitölt, a teljes nevet és az e-mail címet, mivel ezeket már megadta a felhasználó regisztrációnál. A mezők szintén csak 255 karakter hosszúságú szövegek, két oszlop kivételével, az id és a carId. Az **id** az AUTO_INCREMENT attribútum alapján kapja a számát, míg a **carId** az Autók táblából kapja az azonosítóját. Ezekon kívül tartalmaz még **fullname** (teljes név), **email** (email cím), **phonenumber** (telefonszám), **comment** (megjegyzés) mezőket is. Ezek közül egyedül a comment oszlop esetén lehetséges az, hogy üres értéket kap vissza, hisz a felhasználónak ezt nem muszáj kitöltenie.

Részletes feladatspecifikációk, algoritmusok

A szakdolgozatom megírásához HTML, CSS, Javascript, React és Bootstrap nyelveket és keretrendszereket használtam, az adatbázis létrehozásához MySql-t, valamint a weboldalam és az adatbázis közötti kapcsolatot, valamint az API-ok kezelését NodeJs-el kezeltem.

index.jsx

A projekt könyvtárában egy HTML-t hoztam létre, ami tartalmaz egy root id-val ellátott divet, ami magának a React alkalmazásnak a gyökere. Az index.jsx ebbe a div-be tölti be a tartalmat. Ide beimportáltam magát a fő könyvtárat és a StrictMode-ot. A StrictMode segít az alkalmazásban lévő esetleges problémák azonosításában, ez az egész alkalmazásra vonatkozik, mivel az App komponenst teljesen körbe öleli, ezért hívják wrapper komponensnek. Valamint itt van meghívva a Bootstrap globális stílusai, ami az egész oldalra hatással van. A createRoot lehetővé teszi a renderelést, amit egy root nevű konstansban tárolok el.

```
szakdoga > frontend > src > index.jsx > ...
1  import React, {StrictMode} from "react";
2  import {createRoot} from "react-dom/client";
3  import 'bootstrap/dist/css/bootstrap.min.css';
4
5  import App from "./App.jsx";
6
7  const root = createRoot(document.getElementById("root"));
8  root.render(
9    <StrictMode>
10     <App />
11   </StrictMode>
12 );
```

32. ábra index.jsx

App.jsx

Ez az a fájl, amit beimportáltam az index.jsx-be, innen ágazik el az alkalmazás a további oldalakra, például a főoldalra, autóinkra stb. Az oldalon a navigálás itt van meghatározva a BrowserRouter segítségével. Hogy ezt a Router-t tudjuk használni, behívtam ide az összes oldalt és komponenst, és adtam nekik egy elérési utat a Routes segítségével. Valamint itt vannak deklarálva a bejelentkezést, kijelentkezést kezelő függvények.

```

import React, { useState, useEffect } from "react";
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
import "bootstrap/dist/css/bootstrap.min.css";
import "../styles/Index.css";
import { Helmet } from "react-helmet";
import NavbarComp from "../pages/components/Navbar.jsx";
import FooterComp from "../pages/components/Footer.jsx";
import Login from "../Login";
import Register from "../Register";

```

33. ábra Komponensek beimportálása

Az weboldal kezeli a felhasználó hitelesítését `useState`, `useEffect` és `localStorage` segítségével. Az alkalmazás `localStorage`-be menti a token, hogy az oldal frissítése esetén is bejelentkezve maradjon a felhasználó, valamint kijelentkezés esetén törli ebből a tárolóbol a felhasználói adatokat és annak tokenjét.

A felhasználói adatokat a `useEffect` segítségével tölti be, ha van mentett token, akkor lekérdezi a szerverről az adatokat, viszont, ha a válasz nem tartalmaz érvényes felhasználót akkor a token törlődik.

```

useEffect(() => {
  if (token) {
    fetch("http://localhost:3000/user", {
      headers: { Authorization: `Bearer ${token}` },
    })
      .then((response) => response.json())
      .then((data) => {
        if (data.username) {
          setUser(data);
        } else {
          localStorage.removeItem("token");
          setToken(null);
        }
      })
      .catch((error) => {
        console.error("Sikertelen hitelesítés:", error);
        localStorage.removeItem("token");
        setToken(null);
      });
  }
}, [token]);

```

34. ábra Felhasználó betöltése

Backend

A backend szerveremhez a következő csomagokat használtam:

- Argon2: Hasheli a jelszót, ez a bcryptjs modernebb alternatívája
- Cors: Kezeli a böngészők cross-origin kéréseit
- Dotenv: Környezeti változókat kezel .env fájlban (pl.: API kulcsok, jelszavak)
- Express: HTTP szerver létrehozása
- JSON webtoken: Hashelt token generálása és autentikálása
- Mysql2: Adatbáziskapcsolat és promise alapú kérések (async/await)
- Nodemailer: E-mailek küldése a felületről

```
"dependencies": {  
  "argon2": "^0.41.1",  
  "bcryptjs": "^2.4.3",  
  "cors": "^2.8.5",  
  "dotenv": "^16.4.7",  
  "express": "^4.21.2",  
  "jsonwebtoken": "^9.0.2",  
  "mysql2": "^3.12.0",  
  "nodemailer": "^6.10.0"  
}
```

35. ábra Függőségek

A csomagok szükséges beállításait a következőképpen tettem: a **dotenv** csomag betölti a környezeti változókat a .env fájlból. A config() metódus lehetővé teszi az említett változók elérését a kód futása alatt.

Ami még fontos tudnivaló erről a kódrészletről, az az hogy a cors biztosítja a kommunikációt a frontend és a backend között, ezért tartalmaz egy Authorization fejléceket, ami elérhetővé teszi a token a frontend számára.

A következő ábrán pedig a .env beállítása szerepel, ami

JWT_SECRET, EMAIL_USER és EMAIL_PASS változókból áll. Ezt biztonsági okokból kifolyólag nem ajánlott közzétenni. Amit beillesztettem ábra, is csak egy példa, hogy hogyan kéne kinéznie ennek a fájlnek.

```
dotenv.config();  
const app = express();  
app.use(cors({  
  exposedHeaders: ['Authorization']  
}));  
app.use(express.json());  
const PORT = 3000;  
app.use(cors({ origin: 'http://localhost:3001' }));  
  
const db = mysql.createPool({  
  host: 'localhost',  
  user: 'root',  
  password: '',  
  database: 'racedatabase',  
});
```

36. ábra Csomagok beállítása

```
JWT_SECRET = pelda_titkos_kulcs  
EMAIL_USER = emaicim@gmail.com  
EMAIL_PASS = aaaa bbbb cccc dddd
```

37. ábra .env fájl

A `server.js` az említett csomagok importálását és beállításait tartalmazza, ezen kívül itt írtam meg az API-okat, például a regisztrációt, bejelentkezést, autók lekérését, profilszerkesztés, vélemények stb. Példaként bemutatom a `/register` és `/login` végpontokat, valamint a `/reviews-t`, mivel az kicsit eltér a többitől.

`/register`

Ez a végpont egy új felhasználót regisztrál az adatbázisba. Az Adatmodell leírása cím alatt megnézhetik és elolvashatják, hogy mely adatokat fogja ez a kérés küldeni az adatbázisba sikeres regisztráció esetén.

Először is egy `if` függvény segítségével azonosítja, hogy létezik-e már ilyen felhasználónév, ha igen akkor visszadob egy hibát 400-as státuszkóddal és egy üzenettel. Viszont, ha még nincs ilyen felhasználó akkor egy `hashedPassword` konstansban eltárolja a titkosított jelszót, majd később ezt fogja nekünk feltölteni a `password` mezőbe, nem pedig a nyers felhasználó által beírt jelszót. A következő blokk az adatbázisba lévő beszúrás, ahol egy egyszerű SQL paranccsal feltöltjük az adatokat, viszont itt a `Values` kérdőjelekkel van helyettesítve az SQL Injection miatt, és majd csak később vannak megadva az adatok. Ezután jön a `mailOptions`, ami akkor fut le, ha sikeresen regisztrált a felhasználó. Ehhez szükséges létrehozni a `transporter` nevű konstans, ami a `nodemailer.createTransport` segítségével eltárolja azokat az adatokat amelyek segítségével a felület küldi az e-mailt, ezek a `.env` fájlban tárolódnak, mint változók. A `mailOptions`nek megadtam azt, hogy milyen címről jön az e-mail, mire (megadott e-mail cím), mi a levél tárgya és mi az üzenet. Ezalatt még van egy konzolos hibakezelés, ami kiírja a konzolba, hogy sikeres vagy sikertelen volt az e-mail küldése, valamint mivel ez az egész egy `try-catch`-ben van lefuttatva, ezért az aljára került még egy hibakezelés, ami pontosabban a regisztrációhoz tartozik.

```
try {
  const [users] = await db.query("SELECT * FROM users WHERE username = ?", [username]);
  if (users.length > 0) {
    return res.status(400).json({ message: "Már létezik ilyen felhasználónév." });
  }

  const hashedPassword = await argon.hash(password);
```

38. ábra Felhasználó keresése és jelszó hashelése

```
await db.query(
  "INSERT INTO users (username, lastname, firstname, email, password, isAdmin) VALUES (?, ?, ?, ?, ?, 0)",
  [username, lastname, firstname, email, hashedPassword]
);
```

39. ábra Adatok beszúrása adatbázisba

```
const mailOptions = {
  from: "kucsikgabor22@gmail.com",
  to: email,
  subject: "Sikeres regisztráció!",
  text: `Kedves ${username},\n\nÜdvözlünk a Race-001 Autókölcsönző weboldalán! 📧`
};
```

40. ábra mailOptions

```
transporter.sendMail(mailOptions, (error, info) => {
  if (error) {
    console.error("Hiba az e-mail küldése közben!");
  } else {
    console.log(`Az e-mail sikeresen elküldve a ${email} címre.`)
  }
});

res.status(201).json({ message: "Sikeres regisztrálás!" });
} catch (err) {
  console.error("Hiba regisztráció közben: ", err);
  res.status(500).json({ message: "Hiba a regisztráció közben." });
}
});
```

41. ábra Hibakezelések

/login

Ez a végpont egy felhasználói bejelentkezést kezel, de itt már csak két adatot kér le a req.body-ból, ami nem más mint a username és password, hisz ezekkel az adatokkal tudja magát igazolni a felhasználó.

Ezzel az SQL lekérdezéssel az adatbázisból megkeressük az users táblában azt a felhasználót, akinek a username mezője megegyezik a felhasználó által megadott felhasználónévvel.

```
const [users] = await db.query("SELECT * FROM users WHERE username = ?", [username]);
if (users.length === 0) {
  return res.status(400).json({ message: "Nincs ilyen felhasználó." });
}
```

42. ábra Felhasználónév keresése a users táblában

Ha nincsen ilyen felhasználó akkor az ábrán is látható üzenetet fogja küldeni a szerver, miszerint nincs ilyen felhasználó. A következő lépésben ellenőrizzük a jelszavak megegyezését egy isMatch nevű konstans keretein belül. Az argon csomag verify nevű függvényével ezt könnyen megállapíthatjuk. Ezután jön egy if függvény, aminek az a kritériuma, hogyha az isMatch hamis, akkor visszaküldje a felhasználónak azt az üzenetet, hogy nem jó jelszót írt be. Ha a jelszót helyesen beírta a felhasználó akkor létrehozásra kerül egy JWT **token**, ami tartalmazza a felhasználó egyedi azonosítóját, felhasználónevét és az isAdmin logikai változót, amiben tárolva van az, hogy a felhasználó admin-e vagy sem.

Ez a token az „Authorization” fejlécben (header) továbbítható a felhasználónak, amivel később a kéréseit hitelesítheti.

```
const isMatch = await argon.verify(user.password, password);
if (!isMatch) {
  return res.status(400).json({ message: "Hibás jelszó." });
}
```

43. ábra Jelszó ellenőrzése

```
const token = jwt.sign(
  {
    id: user.id,
    username: user.username,
    isAdmin: user.isAdmin === 1 // Itt az adatbázisból olvassuk az admin státuszt
  },
  process.env.JWT_SECRET,
  { expiresIn: "1h" }
);
res.json({ token, isAdmin: user.isAdmin === 1 });
```

44. ábra Token generálása

Vélemény API

Az oldalhoz való vélemények egy review nevű konstansban vannak eltárolva, Ezek teljes mértékben kitalált személyek, kitalált értékelésekkel. Ezek a következő adatokat tartalmazzák:

- id: Vélemény azonosítója
- userName: A vélemény írójának neve
- rating: Értékelés 1-5-ig terjedő skálán
- comment: Szöveges visszajelzés

A vélemények lekérdezése GET kérés küldésével történik a /reviews végpontra. Válaszban a véleményeket JSON formátumban kapjuk meg.

```
{
  "id": 4,
  "userName": "ValyiIstvan",
  "rating": 5,
  "comment": "A Bentley Continental GT Speed egy igazi élmény, minden kanyarban érezni a luxust;"
},
{
  "id": 5,
  "userName": "Race001 Gábor",
  "rating": 5,
  "comment": "A Bentley egyáltalán nem nagypapa autó. Kényelmes és elegáns."
},
}
```

45. ábra Értékelések JSON formátumban

```
app.get("/reviews", (req, res) => {
  res.json(reviews);
});
```

46. ábra Get kérés

Forráskód

Először is, szeretném bemutatni a projekt mappáját, hogy pontosabb belátást kapjanak a weboldal felépítésébe. A szakdolgozatomat két külön mappára osztottam a gyökérkönyvtárban, ezek a backend és frontend, értelemszerűen a backend mappa a backend fájlokat tartalmazza, míg a frontend a frontend fájlokat.

Home.jsx

Ezen az oldalon szerepelnek kártyák, amik egy-egy hivatkozást alkotnak, ezekkel olyan oldalakra lehet navigálni, ahol márka szerint tudjuk megtekinteni az autókat. Ezek a kártyák autó márka képekkel vannak ellátva, hogy el tudjunk rajtuk igazodni. Ezek így néznek ki:

```
<Col xs={4} sm={3} md={2}>
  <Button variant="link" className="car-brands" onClick={() => navigate('/mercedes')}>
    <Image src={require('../img/brandlogos/mercedes.png')} fluid />
  </Button>
</Col>
```

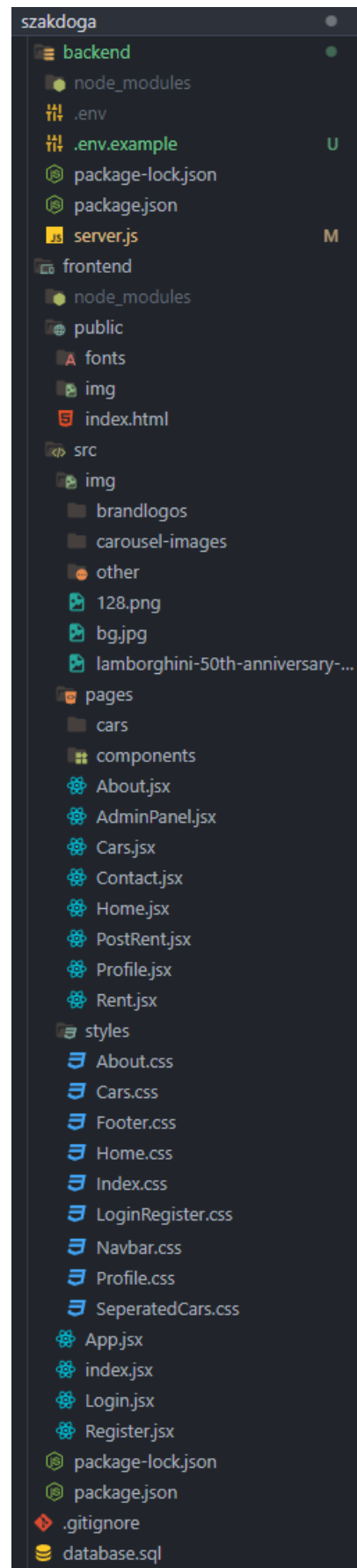
47. ábra Autó kártya példa

A következő elem az egy Carousel képmegjelenítő, ami lehetővé teszi számunkra, hogy oda-vissza tudjuk lapozni a fényképnézegetőt, képleírással ellátva. Ennek az elemnek a képei a forráskód elején vannak importálva, mint Slide1, Slide2 stb.

```
<Carousel.Item>
  <img className="carousel-image d-block w-100" src={Slide3} alt="Mercedes Maybach S680" />
  <Carousel.Caption>
    <h3>Mercedes Maybach S680</h3>
    <p>Exkluzív design, luxus és kényelem egy autóban.</p>
  </Carousel.Caption>
</Carousel.Item>
```

48. ábra A képmegjelenítő egyik eleme

Az oldal alján pedig megtalálhatjuk a véleményeket, amiről már volt szó fentebb, ezek kártyákban kaptak helyet, a véleményt író felhasználó neve erősebben van megjelenítve, hogy eltérjen a megjegyzéstől, illetve értékeléstől. Az értékelést egy bronz árnyalattal jelenítettem meg, míg a megjegyzés egy sima bejegyzés stílust kapott.



46. ábra Könyvtár szerkezete

Navbar.jsx és Footer.jsx komponensek

Ezek a komponensek a pages mappán belül a components mappában kaptak helyet. Mind a kettő fájl az App.jsx-ben vannak behívva, így mindegyik oldalon megjelennek, ezáltal nincsenek eltérések az oldalak között.

A **Navbar** komponens tartalmazza a linkeket, hogy az oldalak között tudjon navigálni a felhasználó, itt van a kedvencek funkció, bejelentkezett állapotban a kosár, profilszerkesztés, valamint a kijelentkezés.

```
const loadFavorites = () => {
  const savedFavorites = JSON.parse(localStorage.getItem('carFavorites')) || {};
  const favoritesArray = Object.keys(savedFavorites)
    .filter(carId => savedFavorites[carId]?.value)
    .map(carId => ({
      id: carId,
      brand: savedFavorites[carId].brand,
      name: savedFavorites[carId].name,
      image_url: savedFavorites[carId].image_url
    }));
  setFavorites(favoritesArray);
};
```

49. ábra Kedvencek betöltése

A Footer komponens egy egész egyszerű láblécezt jelenít meg a weboldal alján. Egyetlen bekezdést tartalmaz a Copyright logóval ellátva. Erre a komponensre inline stílust alkalmaztam, amit a footer tag-nél be is hívtam a style={styles.footer} segítségével. Mivel position-re fixed van beállítva, ezért mindig a lap alján láthatjuk a láblécezt, még akkor is, ha görgetünk az oldalon.

```
function FooterComp(){
  return(
    <footer style={styles.footer}>
      <div>
        <p>&copy;2024 Race-001 Kölcsonzó.</p>
      </div>
    </footer>
  );
};
```

50. ábra Footer visszatérési értéke

Tesztelési Dokumentáció

Megkértem pár családtagot, hogy teszteljék le a weboldalt, hogy jobb betekintést kapjak az általános felhasználói élménybe, illetve meghallgassam a visszajelzéseket az oldal működéséről. A projekt készítése alatt számos hibába ütköztem, de ezeknek a nagy részét sikerült kijavítani.

Meg szeretném említeni a **kosár** funkciót, mivel a Törlés és Meghosszabbítás gombok nem működnek, illetve a kölcsönzött tétel az oldal frissítése után érhető el a legördülő menüben. Ugyanígy a kosár és a kedvencek menüpontokkal azt tapasztaltam, hogy különböző ablak méretekben nem úgy néznek ki a menüben a képek, mint én azt szerettem volna.

Továbbfejlesztési lehetőségek

Design

Nagyon szerettem volna, ha a projekt olyan formában nyilvánul meg, mint ahogy én azt szerettem volna. Ezért ezt sajnos csak ide tudom sorolni, ha van a weboldalnak egy megfelelő kinézete, akkor teljes mértékben megállhatná a helyét élesben is.

Kedvenc funkció

Az autók kedvencek közé válogatása csak az Autók fülön működik, a márka alapján szeparált oldalakon már nem.

Autók elkülönítése

Mint említettem a márkaszerinti oldalakat, ahol külön vannak megjelenítve az autók, erre lehetne egy jobb megoldást írni, hogy kedvezzen az oldal a tiszta kód elvének.

Fizetés

Lehetne csinálni a kölcsönzéshez egy Fizetés oldalt, ahol a felhasználó ki tudja választani, hogy mivel, szeretne fizetni és hogyan. Ezt akár a Rent.jsx-be is meglehetne csinálni, ahol a felhasználó megadja a fizetési adatait.

Kölcsönzött autók

A kosár legördülő menüjében helyet kapott egy Törlés és Meghosszabbítás gomb, amik sajnos nem funkcionálnak. Ezt jó lenne megvalósítani a jövőben, hogy a felhasználó igazán kedvére tudja szabni a rendelését.

Összegzés

Ez a projekt megváltoztatta a nézeteimet a programozás iránt, és rájöttem arra, hogy új dolgokat tanulni nem olyan nehéz. Ez nem azt jelenti, hogy 3 hónap alatt profi lesz az ember egy új programozási nyelv, keretrendszer használatában, hanem azt, hogy jó látni a fejlődést az idő múlásával. Amit régen elveszítettem lelkesedést, az úgy érzem, hogy most kezd visszatérni.

Részletesebb betekintést kaptam mind a frontend és backend fejlesztésbe is, és rájöttem, hogy nincs könnyebb vagy nehezebb választás, a saját akarattól függ, hogy ki mennyit hoz ki belőle. A záródolgozat segített abban, hogy felismerjem milyen téren kell még fejlesztenem magam és mit kell előtérbe helyezzek. Adott egy olyan képet, amely megmutatja a használt technológiák terén melyekre kell több időt szánnom, hogy sikeres legyek a szakmámban. A React, mint teljesen új frontend keretrendszer meglepetés volt számomra, mivel számos beépített technológiát és modult tartalmaz, amelyek óriási segítséget nyújtanak egy az enyémhez hasonló weboldal elkészítésében. Ritkán éreztem azt az elmúlt időben, hogy a saját örömeimre programozok, de eme projekt segítségével néhol újra átélhettem ezt az érzést. Nagyon jó visszatekinteni arra az útra, amin sikerült elérnem idáig, a HTML alapoktól kezdve, a JavaScript-en keresztül mostanáig. Ami számomra még újnak mondható, az a NodeJs, mint szerveroldali technológia. Ez egy JavaScript alapú futtatókörnyezet, ami lehetővé teszi számunkra, hogy létrehozzunk egy lokális webszervert.

Az oktatók nagyon sok segítséget nyújtottak az elmúlt időszakban és mindig tudtak választ adni a kérdéseimre. Megtanultam mennyire fontos tényező az idő beosztása, nem csak a programozásban vagy egy beadandó esetén, hanem összességében az életben.

Források

Dokumentációk, források

<https://react.dev/learn>

<https://react-bootstrap.netlify.app/>

<https://nodejs.org/en>

<https://www.npmjs.com/>

<https://stackoverflow.com/>

Képek

<https://www.jamesedition.com/cars>

Adatbázisban, image_url mezőben szereplő képcímek